

INTRODUCTION TO CRYPTOGRAPHY

[Kaynak](#)

İÇERİK

1. Temel Kriptografi Kavramları ve Caesar Şifrelemesi
2. Simetrik Şifreleme
3. Asimetrik Şifreleme ve RSA
4. Diffie-Hellman Anahtar Değişimi
5. Hashing & HMAC
6. PKI (Public Key Infrastructure) ve SSL/TLS
7. Password Authentication & Storage (Parola Doğrulama ve Saklama)
8. HTTPS & Login Flow: Putting It All Together (Örnek Senaryo)

Temel Kriptografi Kavramları ve Caesar Şifreleme

1. Kriptografinin Temel Amacı

Kriptografinin özü, bir mesajı (plaintext/açık metin) sadece belirli bir anahtara veya bilgiye sahip olan kişinin çözebileceği bir karmaşaya (ciphertext/şifreli metin) dönüştürmektir. Eğer bir mesajı gönderirken araya bir saldırgan girerse, elindeki veri anlamsız bir gürültüden ibaret kalmalıdır.

Bu süreçte karşımıza çıkacak ana başlıklar şunlardır:

- **Symmetric Encryption (Simetrik Şifreleme):** Örn. **AES**. Şifreleme ve deşifreleme için aynı anahtar kullanılır.
- **Asymmetric Encryption (Asimetrik Şifreleme):** Örn. **RSA**. Birbiriyle matematiksel olarak bağlı bir genel (public) ve bir özel (private) anahtar çifti kullanılır.
- **Diffie-Hellman Key Exchange:** Güvensiz bir kanal üzerinden (internet gibi) iki tarafın ortak bir gizli anahtar üzerinde anlaşmasını sağlayan yöntem.
- **Hashing (Özetleme):** Verinin parmak izini çıkarma işlemidir; şifrelemeden farkı tek yönlü olmasıdır (geri döndürülemez).
- **PKI (Public Key Infrastructure):** Dijital sertifikaları ve anahtarları yönetmek için kullanılan sistemler bütünüdür.

2. Klasik Bir Yaklaşım: Caesar Cipher (Sezar Şifrelemesi)

Kriptografi tarihinin en eski ve en basit yöntemlerinden biri olan **Caesar Cipher**, yaklaşık 2000 yıl önce kullanılmıştır. Bu yöntem bir **Substitution Cipher** (Yerine Koyma Şifrelemesi) türüdür.

Çalışma Mantığı

Sezar şifrelemesi, alfabedeki her harfi belirli bir sayı kadar sağa veya sola kaydırarak çalışır.

- **Key (Anahtar):** Harflerin kaç pozisyon kaydırılacağını belirleyen sayıdır.
- **Encryption (Şifreleme):** Eğer anahtarımız **3** ise ve sağa kaydırma yapıyorsak; **A** harfi **D** olur, **B** harfi **E** olur.
- **Decryption (Deşifreleme):** Alıcı, mesajı çözmek için kaydırma miktarını bilmek zorundadır. Şifrelenmiş metni tersi yönde (3 birim sola) kaydırarak orijinal metne ulaşır.

Örnek: "TRY HACK ME" ifadesini **3** anahtarıyla şifreleyelim:

- T -> W
- R -> U
- Y -> B
- ...sonuç: **"WUB KDFN PH"**

Keyspace (Anahtar Uzayı) Analizi

Bir şifreleme sisteminin güvenliği, denenebilecek toplam anahtar sayısına bağlıdır. Sezar şifrelemesinde:

- İngiliz alfabesinde 26 harf olduğu için, anahtar **1 ile 25** arasında bir değer alabilir.
- **Anahtar 0:** Hiçbir değişiklik yapmaz.
- **Anahtar 26:** Tam bir tur döndüğü için metin yine kendisine dönüşür.
- Bu durumda **Keyspace** sadece **25**'tir. Modern sistemlerle kıyaslandığında bu sayı gülünç derecede küçüktür.

3. Brute Force (Kaba Kuvvet) ile Şifre Kırma

Eğer elimizde şifrelenmiş bir metin varsa ve anahtarı (kaydırma miktarını) bilmiyorsak, Sezar şifrelemesini kırmak çok kolaydır. Çünkü denememiz gereken sadece 25 olasılık vardır. Tüm ihtimalleri tek tek deneme işlemine **Brute Force** saldırısı diyoruz.

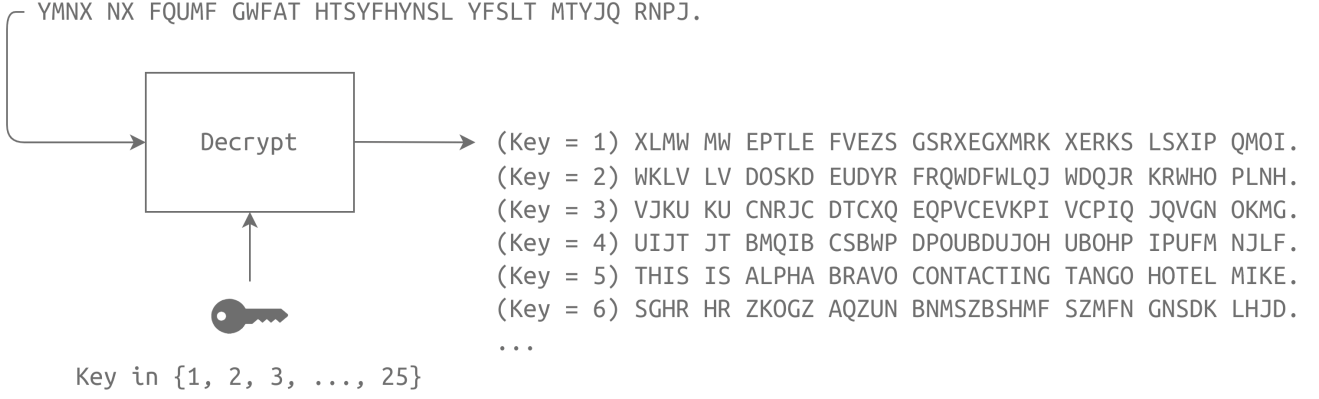
Senaryo: Ele geçirilen şifreli mesaj: YMNX NX FQUMF GWFAT HTSYFHYNLS YFSLT MTYJQ RNPJ

Bu mesajı çözmek için 1'den başlayarak tüm kaydırma kombinasyonlarını deneriz:

1. **Key 1:** "XLMW MW..." (Anlamsız)
2. **Key 2:** "WKLW LV..." (Anlamsız)
3. ...
4. **Key 5:** Denendiğinde karşımıza şu metin çıkar:

"THIS IS ALPHA BRAVO CONTACTING TANGO HOTEL MIKE"

Görüldüğü üzere, **Key 5** kullanıldığında anlamlı bir İngilizce cümle elde ettik. Bu, şifreleme sisteminin zayıflığını ve anahtar uzayının ne kadar dar olduğunu kanıtlar.



Yerine Koyma ve Yer Değiştirme Şifrelemeleri: Substitution vs. Transposition

Kriptografi dünyasında veriyi karıştırmak için kullanılan iki temel yaklaşım vardır. Önceki bölümde gördüğümüz Caesar Cipher bir **Substitution Cipher** (Yerine Koyma Şifrelemesi) örneğiydi. Şimdi ise yapıyı tamamen değiştiren **Transposition Cipher** (Yer Değiştirme Şifrelemesi) mantığını inceleyeceğiz.

1. Temel Fark: İçerik mi Değişiyor, Yer mi?

- **Substitution Cipher (Yerine Koyma):** Alfabadeki bir harfin yerine başka bir harf gelir. Harflerin sırası/yeri değişmez, sadece kimlikleri değişir.
- **Transposition Cipher (Yer Değiştirme):** Mesajdaki harfler aynı kalır, ancak bu harflerin diziliş sırası karıştırılır. Yani içerik sabit, konum değişkendir.

2. Transposition Cipher (Yer Değiştirme) Çalışma Mekanizması

Bu yöntemi anlamak için bir tablo mantığı kullanırız. Süreç genelde "sütunlara yaz, satırlardan oku" prensibiyle ilerler.

Örnek Senaryo:

- **Açık Metin (Plaintext):** THIS IS ALPHA BRAVO CONTACTING TANGO HOTEL MIKE
- **Anahtar (Key):** 42351
- **Ön Hazırlık:** Şifreleme işlemine başlamadan önce metindeki tüm boşluklar kaldırılır.

Adım 1: Sütunlara Yazma (Writing by Columns)

Anahtarımız 5 haneli (42351) olduğu için 5 sütunlu bir tablo oluşturuyoruz. Mesajın harflerini yukarıdan aşağıya doğru sütun sütun dolduruyoruz:

1. **Sütun (Key 4):** T, H, I, S... şeklinde mesajın ilk harfleri aşağı doğru dizilir.
2. **Sütun (Key 2):** Mesajın devamındaki harfler buraya doldurulur.

3. ...ve bu işlem tüm mesaj bitene kadar devam eder.

Adım 2: Sütunları Yeniden Düzenleme (Rearranging)

Tablo dolduktan sonra, sütunlar anahtardaki rakamlara göre yer değiştirir. Örneğin; anahtar 42351 ise:

- Asıl 1. sütun, tablonun 5. sırasına (anahtardaki 1'in konumuna) taşınır.
- Asıl 5. sütun, tablonun 4. sırasına (anahtardaki 5'in konumuna) taşınır.

Adım 3: Satır Satır Okuma (Reading by Rows)

Sütunlar anahtara göre dizildikten sonra, şifreli metni elde etmek için tabloyu soldan sağa doğru **satır satır** okuruz.

- Sonuç (Ciphertext):** NPCOTGHOTH... şeklinde devam eden, harfleri orijinal metindekiyle aynı olan ama sırası tamamen bozulmuş bir metin ortaya çıkar.

1	2	3	4	5
T	P	C	N	O
H	H	O	G	T
I	A	N	T	E
S	B	T	A	L
I	R	A	N	M
S	A	C	G	I
A	V	T	O	K
L	O	I	H	E

Plaintext

THIS IS ALPHA BRAVO
CONTACTING TANGO HOTEL MIKE

4	2	3	5	1
N	P	C	O	T
G	H	O	T	H
T	A	N	E	I
A	B	T	L	S
N	R	A	M	I
G	A	C	I	S
O	V	T	K	A
H	O	I	E	L

Ciphertext

NPCOTGHOTHANEIABTLS
NRAMIGACISOVTKAHOIEL

Önemli Analiz ve Karşılaştırma

Özellik	Substitution (Caesar vb.)	Transposition (Yer Değiştirme)
Harf Kimliği	Değişir (A olur D)	Aynı kalır (A yine A'dır)
Harf Konumu	Aynı kalır	Değişir (1. harf 10. harf olabilir)
Zayıflık	Frekans analiziyle (en çok geçen harf E'dir gibi) kolayca çözülür.	Harf frekansları değişmez; sadece doğru "geometriyi" veya anahtarı bulmak gerekir.

Kritik Not: Modern şifreleme algoritmaları (AES gibi), veriyi güvenli hale getirmek için hem **substitution** hem de **transposition** tekniklerini birleştirerek çok katmanlı bir yapı (SP-Network) kullanır. Tek başına ikisi de modern bilgisayarlar için kolay hedeflerdir.

Bir Şifreleme Algoritması Ne Zaman "Güvenli" Sayılır?

Siber güvenlikte "güvenli" kavramı mutlak değildir, **zamana ve işlem gücüne** bağlıdır. Temel kural şudur: Eğer saldırganın orijinal mesajı (Plaintext) geri getirmesi "imkansıza yakınsa" (infeasible), o algoritma güvenlidir.

Bunu matematiksel ve pratik olarak ikiye ayırıyoruz:

Matematiksel Tanım: "Hard Problem"

Bir şifreleme algoritmasının güvenli olması için **Hard Problem** (Zor Problem) kategorisine girmesi gerekir.

- Polynomial Time (Polinom Zamanı):** Bir problemin makul bir sürede çözülebilmesi demektir. Eğer bir problemi büyük girdilerle bile makul sürede çözebiliyorsak, bu "feasible" (yapılabilir) bir işlemdir.
- Güvenli Algoritma:** Polinom zamanında çözülemeyen, çözümü için gereken işlem gücünün ve süresinin mevcut teknolojiyi aştığı problemlerdir.

Pratik Tanım: Zaman Kriteri

- Güvensiz:** Şifrelenmiş bir mesaj **1 hafta** içinde kırılıbiliyorsa, bu yöntem güvensizdir.
- Pratik Olarak Güvenli (Practically Secure):** Eğer aynı mesajı kırmak **1 milyon yıl** sürecekse, bu yöntem güvenli kabul edilir. (Kimse bir mesajı okumak için 1 milyon yıl beklemez).

Mono-Alphabetic Substitution Cipher (Tek Alfabeli Yer Değiştirme Şifresi)

Bu yöntem, alfabedeki her harfin başka bir harfle eşleştirilmesi mantığına dayanır. Basit gibi görünse de anahtar uzayı (Key Space) devasadır.

Çalışma Mantığı

İngilizce alfabesini (26 harf) düşünelim:

- 'a' harfi için 26 olası karşılık seçilir.
- 'b' harfi için kalan 25 harften biri seçilir.
- 'c' harfi için kalan 24 harften biri seçilir.
- Bu işlem 'z'ye kadar devam eder.

Örnek Eşleşme (Mapping):

- Orijinal Alfabe:** abcdefghijklmnopqrstuvwxyz
- Anahtar (Key):** xpatvrzyjhecsdikbfwunqgmoL

Bu anahtara göre:

- Plaintext'teki 'a' → Ciphertext'te 'x' olur.
- Plaintext'teki 'b' → Ciphertext'te 'p' olur.

Önemli: Mesajı alan kişinin (recipient), şifreli metni (Ciphertext) çözebilmesi için elinde bu xpatvrzyjhecsdikbfwunqgmoL anahtarının aynısının olması gerekir.

Neden Kırılabilir? (The Vulnerability: Frequency Analysis)

İlk bakışta bu yöntem çok güvenli durabilir. Çünkü 26 faktöriyel (26!) inanılmaz büyük bir sayıdır ve saldırganın tüm olası anahtarları tek tek denemesi (Brute-Force) imkansızdır.

ANCAK, dili ve istatistiği kullanarak bu şifreleri saniyeler içinde kırabiliriz. Zayıf nokta: **Letter Frequency (Harf Sıklığı)**.

Her dilin kendine has bir parmak izi vardır. İngilizce metinlerde bazı harfler diğerlerinden çok daha sık kullanılır:

İngilizce İstatistikleri (Kritik Veriler)

1. En Sık Geçen Harfler:

- 'e': %13
- 't': %9.1
- 'a': %8.2

2. Kelime Başında En Sık Bulunan Harfler:

- 't': %16
- 'a': %11.7
- 'o': %7.6

Saldırı Mantığı

Saldırgan şifreli metne bakar. Eğer metinde en çok geçen harf 'x' ise ve metin yeterince uzunsa, İngilizce istatistiklerine göre bu 'x' harfinin orijinal metindeki 'e' olma ihtimali çok yüksektir. Ayrıca kelimelerin sözlük yapısı (Dictionary Words) da işin içine girince, şifre çorap söküğü gibi gelir.

4. Pratik Uygulama: Şifreyi Kırmak

Elimizde anahtarı (key) olmayan şöyle bir şifreli metin (Ciphertext) olsun:

Plaintext

```
Uyv sxd gyi siqvw x sinduxjd pvzjdw po axffojdz xgxw wsxcc wuidvw.
```

Bu şifreyi çözmek için anahtarı bilmemize gerek yok. İstatistiksel analiz yapan araçlar kullanacağız.

Araç: quipqiup

CTF'lerde substitution cipher çözmek için en popüler ve güçlü web tabanlı araç **quipqiup**'tir.

- **Web Sitesi:** quipqiup.com
- **Yöntem:** Şifreli metni yapıştır → "Solve" butonuna bas.

Sonuç

Araç, harf frekanslarını ve kelime dağarcığını kullanarak saniyeler içinde şu sonucu verir:

Plaintext:

"The man who moves a mountain begins by carrying away small stones."

Çıkarım

Bu örnek, **Mono-Alphabetic Substitution Cipher** yönteminin **kırılmış (broken)** bir algoritma olduğunu kanıtlar. Bu yöntem, gizlilik (confidentiality) gerektiren hiçbir iletişimde **kullanılmamalıdır**.

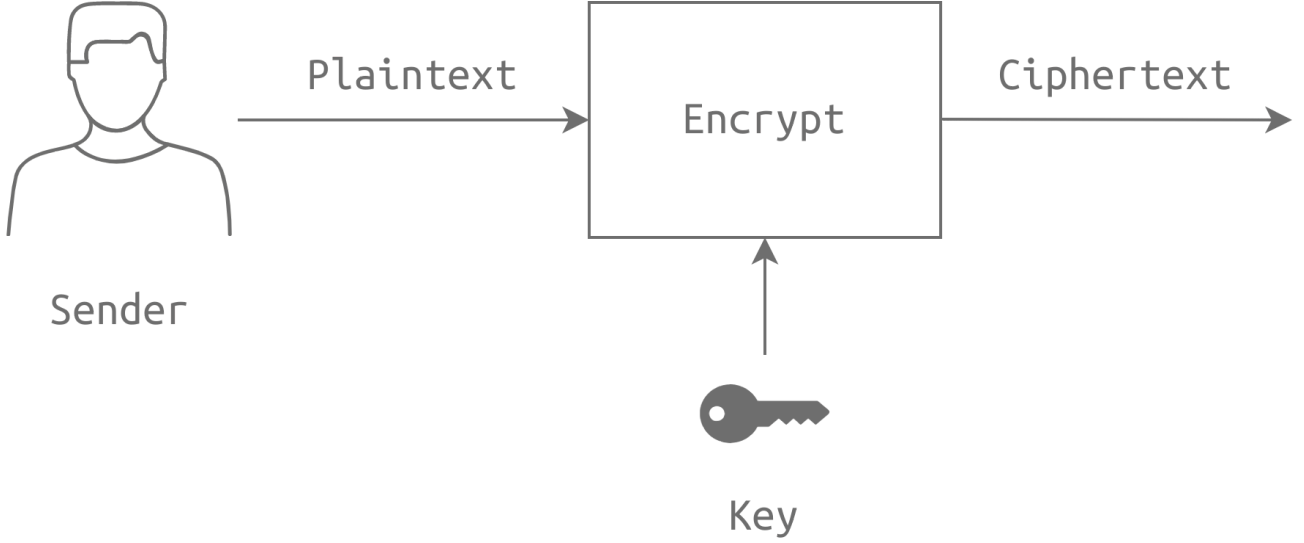
Simetrik Şifreleme

1. Temel Terminoloji (Review)

Şifrelemeye girmeden önce şu 4 terimi adımımız gibi bilmeliyiz:

- **Plaintext (Düz Metin):** Şifrelenecek orijinal mesaj.
- **Ciphertext (Şifreli Metin):** Şifreleme işleminden sonra çıkan okunamaz veri.
- **Cipher (Algoritma):** Şifreleme ve şifre çözme işlemlerini tanımlayan matematiksel kurallar bütünü.

- **Key (Anahtar):** Algoritmanın plaintext'i ciphertext'e (veya tam tersi) çevirmek için kullandığı gizli parametre.

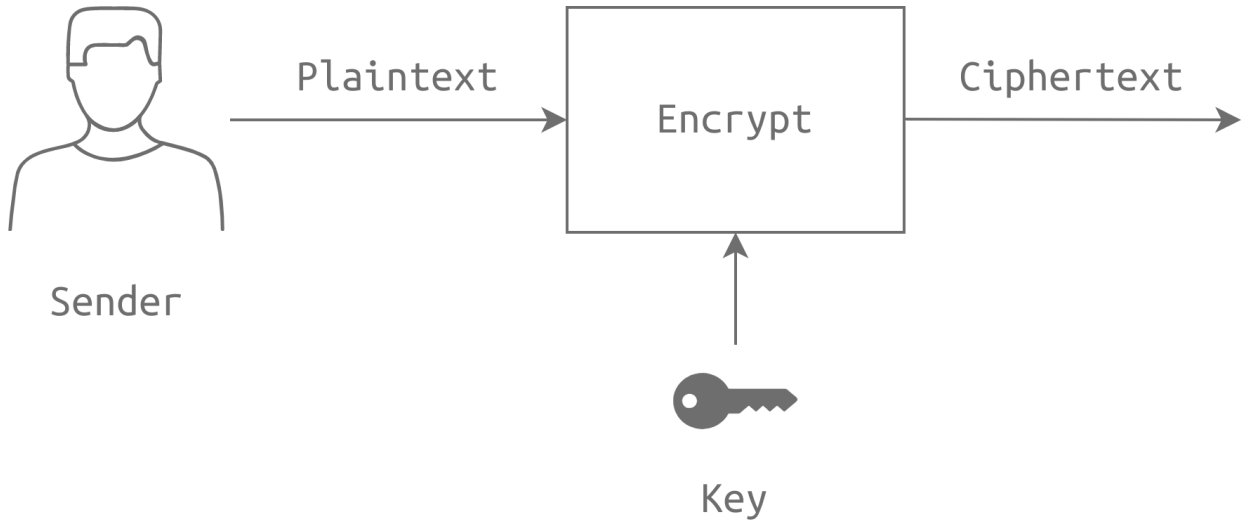


2. Simetrik Şifreleme Nedir?

Simetrik şifrelemenin olayı şudur: **Şifreleme (Encryption)** ve **Şifre Çözme (Decryption)** için **AYNI ANAHTAR** kullanılır.

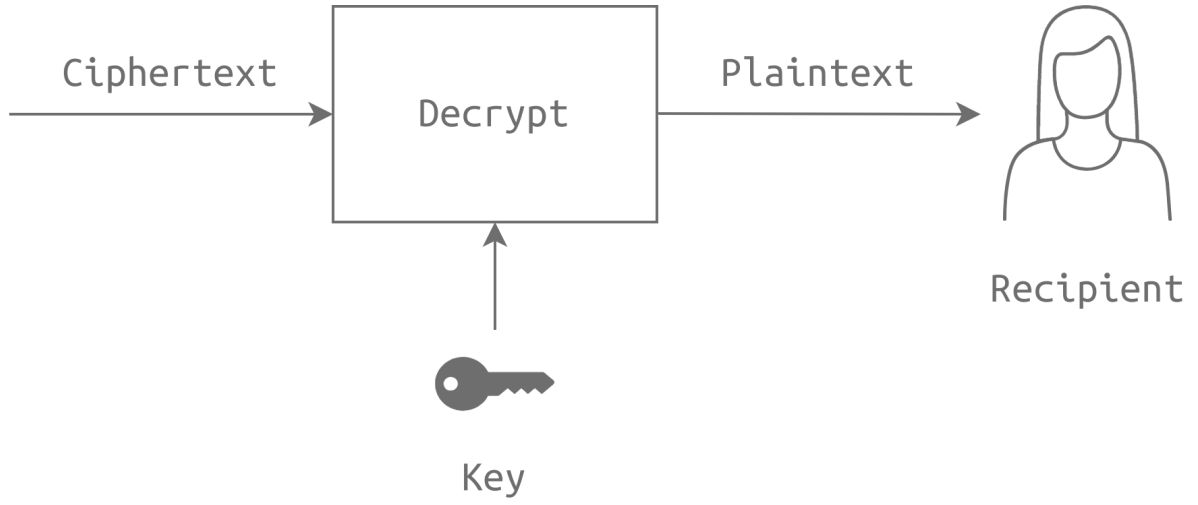
- **Mantık:** Gönderici (Sender) ve Alıcı (Recipient) önceden gizli bir anahtar üzerinde anlaşır.
- **Akış:**

1. Gönderici: Plaintext + Key → Encrypt → Ciphertext



2. İletişim Kanalı: Ciphertext gönderilir.

3. Alıcı: Ciphertext + Key → Decrypt → Plaintext



- **Risk:** Eğer anahtar ele geçirilirse, tüm iletişim ifşa olur. Anahtarın güvenli bir şekilde karşı tarafa iletilmesi (Key Exchange) en büyük zorluktur.

3. Algoritmaların Evrimi: DES'ten AES'e

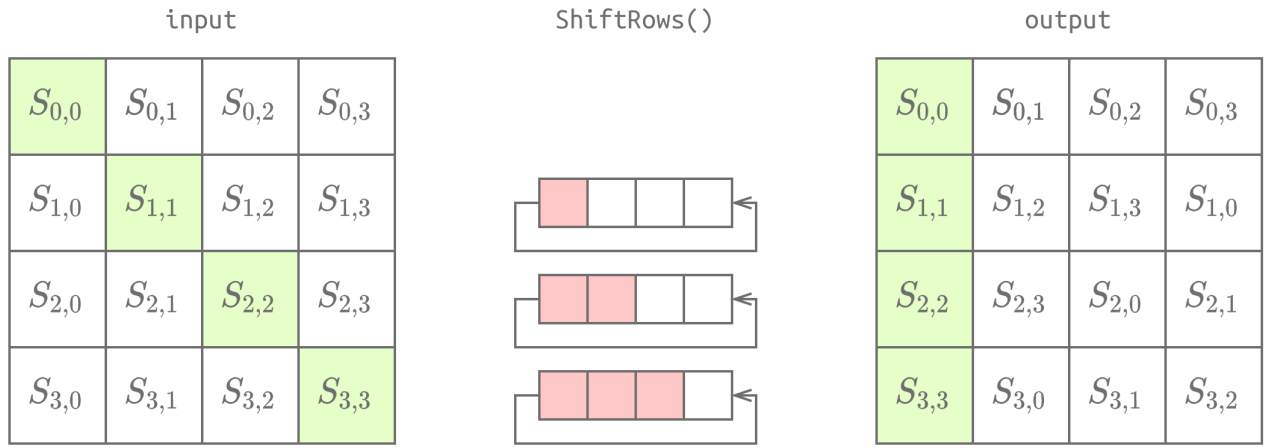
Eskiden ne kullanılıyordu, şimdi ne kullanıyoruz? Burası mülakatlarda veya CTF'lerde çok sorulur.

A. DES (Data Encryption Standard) - Tarih Oldu

- **Çıkış:** 1977 (NIST tarafından).
- **Anahtar Boyutu:** 56-bit.
- **Durum:** GÜVENSİZ (INSECURE).
 - 1998'de bir DES anahtarı 56 saatte kırıldı. Bugün saniyeler sürer.
 - Brute-force (kaba kuvvet) saldırılarına karşı dayanıksızdır.

B. AES (Advanced Encryption Standard) - Modern Standart

- **Çıkış:** 2001 (NIST tarafından).
- **Anahtar Boyutu:** 128, 192 veya 256-bit.
- **Durum:** GÜVENLİ. Bugünün standardıdır.
- **Çalışma Mantığı:** Veriyi 128-bitlik bloklar halinde işler ve anahtar boyutuna göre belirli sayıda "Round" (tur) uygular. Her turda şu 4 işlem yapılır (Detayları ezberlemeye gerek yok, mantığı bil yeter):
 1. **SubBytes:** Byte'ları bir tabloya (S-box) göre değiştirir.
 2. **ShiftRows:** Satırları kaydırır.
 3. **MixColumns:** Sütunları karıştırır/matrisle çarpar.
 4. **AddRoundKey:** Round anahtarı ile XOR işlemi yapar.



C. Diğer Simetrik Algoritmalar (GPG Destekli)

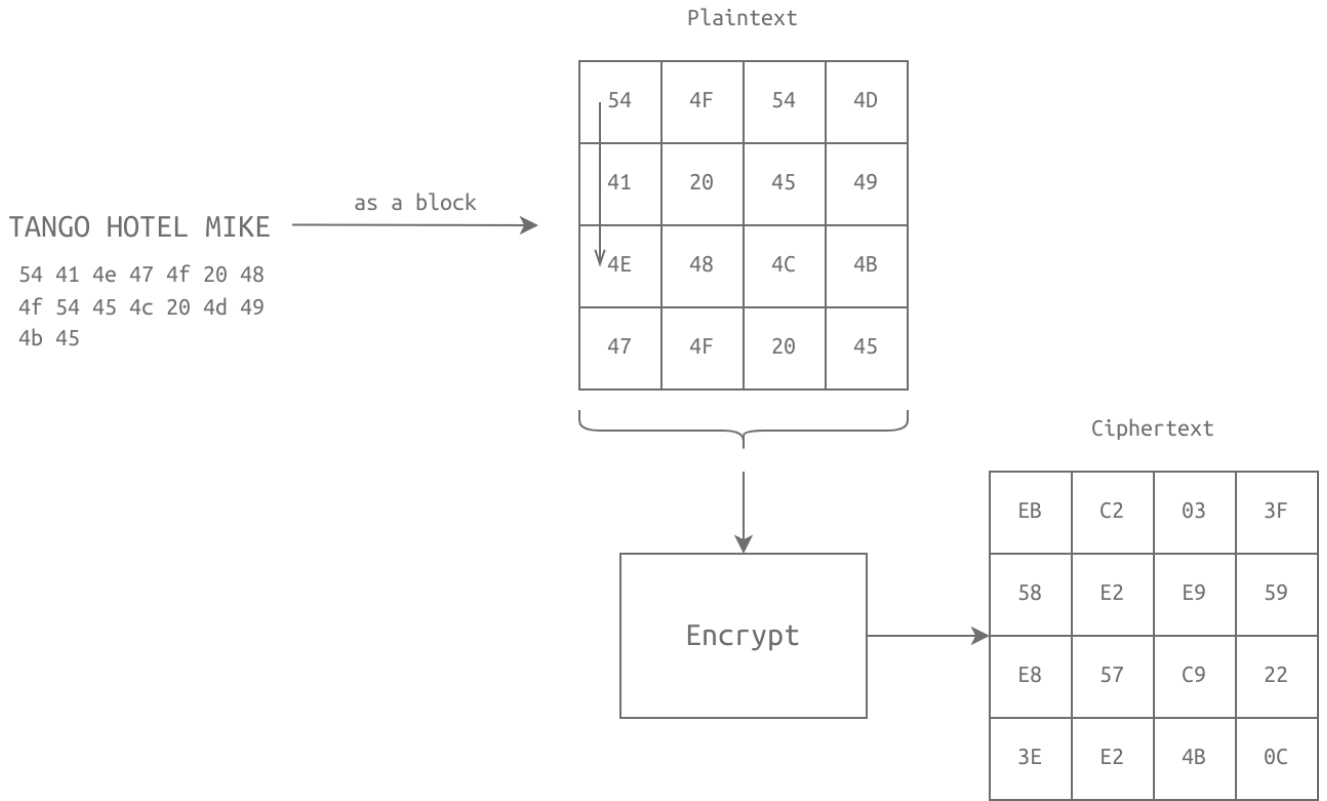
- **IDEA:** International Data Encryption Algorithm.
- **3DES (Triple DES):** DES'in 3 kere uygulanmış hali. Güvenli sayılsa da yavaş ve 2023 itibarıyla kullanımdan kaldırılıyor (deprecated).
- **Blowfish / Twofish:** Bruce Schneier tarafından tasarlandı. AES alternatifi güçlü algoritmalar.
- **Camellia:** Japonya kökenli (Mitsubishi/NTT), güvenli bir algoritma.

4. Block Cipher vs. Stream Cipher

Simetrik algoritmalar veriyi nasıl işlediğine göre ikiye ayrılır:

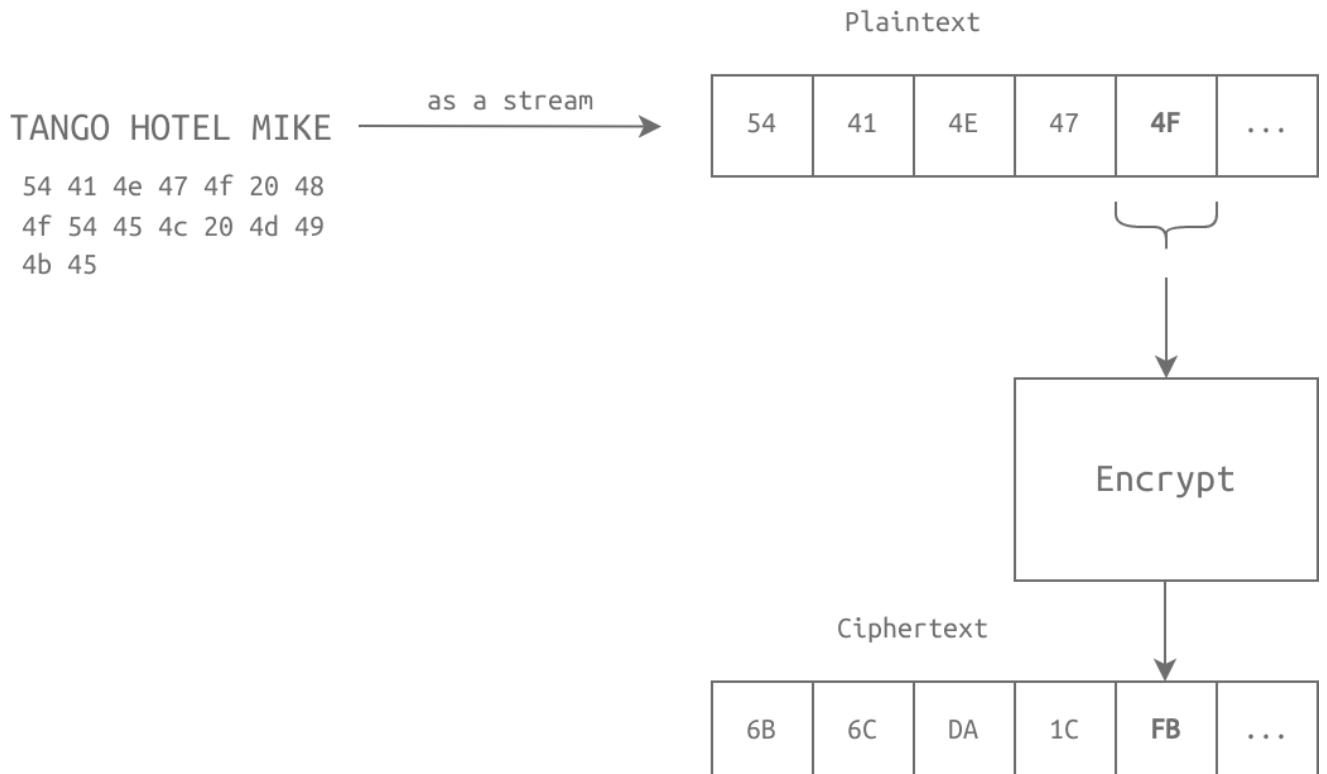
1. Block Cipher (Blok Şifreleme)

- Veriyi sabit boyutlu bloklara (genelde **128-bit** yani **16 Byte**) böler.
- Her blok ayrı ayrı şifrelenir.
- **Örnek:** AES, DES, Twofish.
- **Örnek Senaryo:** "TANGO HOTEL MIKE" (16 karakter) → ASCII Hex'e çevrilir → 4x4'lük bir matrise (16 byte) oturtulur ve tek seferde şifrelenir.



2. Stream Cipher (Akış Şifreleme)

- Veriyi byte-byte (veya bit-bit) şifreler.
- Veri akarken (streaming) şifreleme yapmak için idealdir.
- **Örnek:** RC4 (eski), ChaCha20.
- **Örnek Senaryo:** "TANGO..." → Önce 'T' şifrelenir, gönderilir. Sonra 'A' şifrelenir, gönderilir.



5. Güvenlik Prensipleri (Ne Sağlıyor?)

Simetrik şifreleme doğru kullanıldığında (Güçlü algoritma + Güvenli saklanan anahtar), CIA üçlüsünü sağlar:

1. **Confidentiality (Gizlilik):** Araya giren kişi (Eve) şifreli metni görse de okuyamaz.
2. **Integrity (Bütünlük):** Modern algoritmalarla şifrelenmiş metinde en ufak bir değişiklik (bit flip), şifre çözüldüğünde tamamen anlamsız (gibberish) bir çıktı verir veya şifre çözme hatası verir. Böylece verinin yolda değiştirildiği anlaşılır.
3. **Authenticity (Kimlik Doğrulama):** Mesajı başarıyla çözebiliyorsam, bu mesajı sadece anahtara sahip olan kişi (örneğin Alice) yazmış olabilir. Başkası yazamaz çünkü anahtarı bilmiyor.

6. Ölçeklenebilirlik Sorunu (The Scaling Problem)

Simetrik şifrelemenin en büyük problemi **Anahtar Yönetimidir**.

- 2 Kişi (Alice, Bob) → 1 Anahtar.
- 3 Kişi (Alice, Bob, Charlie) → 3 Anahtar (A-B, A-C, B-C).
- **100 Kişi** iletişim kuracaksa kaç anahtar gerekir?

$$\times (-1)^2 = 100 \times 99 = 9900$$

Bu yönetilemez bir durumdur. Eğer bir kullanıcının güvenliği ihlal edilirse, o kullanıcıyla ilişkili 99 anahtarın yenilenmesi gerekir. **Çözüm:** Asimetrik Şifreleme (sonraki konu).

7. Pratik Uygulama: GPG ve OpenSSL

Linux'ta (AttackBox) bu işleri nasıl yapıyoruz? Bu komutlar CTF'lerde hayat kurtarır.

A. GPG (GNU Privacy Guard)

OpenPGP standardını kullanır.

Şifreleme (Encryption):

Bash

```
gpg --symmetric --cipher-algo AES256 message.txt
```

- `--symmetric` : Simetrik şifreleme yapacağımızı belirtir (parola soracaktır).
- `--cipher-algo AES256` : Algoritmayı seçeriz (AES256 önerilir).
- **Çıktı:** `message.txt.gpg` (Binary format, okunamaz).

ASCII Armored Çıktı (Metin formatında şifreli dosya):

Bash

```
gpg --armor --symmetric --cipher-algo AES256 message.txt
```

- `--armor` : Çıktıyı binary yerine ASCII metin formatında verir (mail atmak için ideal).

Şifre Çözme (Decryption):

Bash

```
gpg --output original_message.txt --decrypt message.gpg
```

- Parolayı girince orijinal dosya oluşur.

B. OpenSSL

GPG'ye göre daha "raw" (ham) bir araçtır.

Şifreleme:

Bash

```
openssl aes-256-cbc -e -in message.txt -out encrypted_message
```

- `aes-256-cbc` : Algoritma ve mod (AES 256-bit, CBC modu).
- `-e` : Encrypt (Şifrele).
- `-in` : Girdi dosyası.
- `-out` : Çıktı dosyası.

Şifre Çözme:

Bash

```
openssl aes-256-cbc -d -in encrypted_message -out original_message.txt
```

- `-d` : Decrypt (Şifre Çöz).

GÜVENLİK NOTU (PBKDF2 Kullanımı):

Varsayılan OpenSSL komutları bazen brute-force'a karşı zayıf olabilir. Anahtarı paroladan türetirken işlemi yavaşlatmak (hardening) için `PBKDF2` (Password-Based Key Derivation Function 2) ve iterasyon sayısı ekleriz:

Bash

```
# Şifrelerken (Daha Güvenli):  
openssl aes-256-cbc -pbkdf2 -iter 10000 -e -in message.txt -out  
encrypted_message
```

```
# Çözerken (Aynı parametreler şart):  
openssl aes-256-cbc -pbkdf2 -iter 10000 -d -in encrypted_message -out  
original_message.txt
```

- `-pbkdf2` : Anahtar türetme fonksiyonunu aktif eder.
- `-iter 10000` : Parolayı anahtara çevirirken 10.000 kez işlem yapar (Saldırganın işini zorlaştırır).

Asimetrik Şifreleme ve RSA

1. Neden Asimetrik Şifreleme? (Symmetric vs Asymmetric)

Simetrik şifrelemede anahtarı karşıya göndermek için **Secure Channel (Güvenli Kanal)** şarttır. Yani hattın dinlenmediğinden (Confidentiality) ve verinin değiştirilmediğinden (Integrity) emin olmalıyız.

Asimetrik şifrelemede ise **Reliable Channel (Güvenilir Kanal)** yeterlidir. Sadece verinin bütünlüğünün (Integrity) korunması yeterlidir; hattın dinlenmesi sorun teşkil etmez çünkü anahtar paylaşımı mantığı tamamen farklıdır.

Key Pair (Anahtar Çifti) Mantığı

Asimetrik şifrelemede **iki adet** anahtar üretilir:

1. **Public Key (Açık Anahtar)**: Herkesle paylaşılır. Web sitesine koyulur, veri tabanlarına yüklenir.
2. **Private Key (Gizli Anahtar)**: **ASLA** paylaşılmaz. Sadece sahibinde kalır ve çok sıkı korunmalıdır.

Altın Kural: Bir anahtarla şifrelenen veri, **sadece** diğeriyle çözülebilir. Matematiksel olarak Public Key'den Private Key'i türetmek imkansızdır (infeasible).

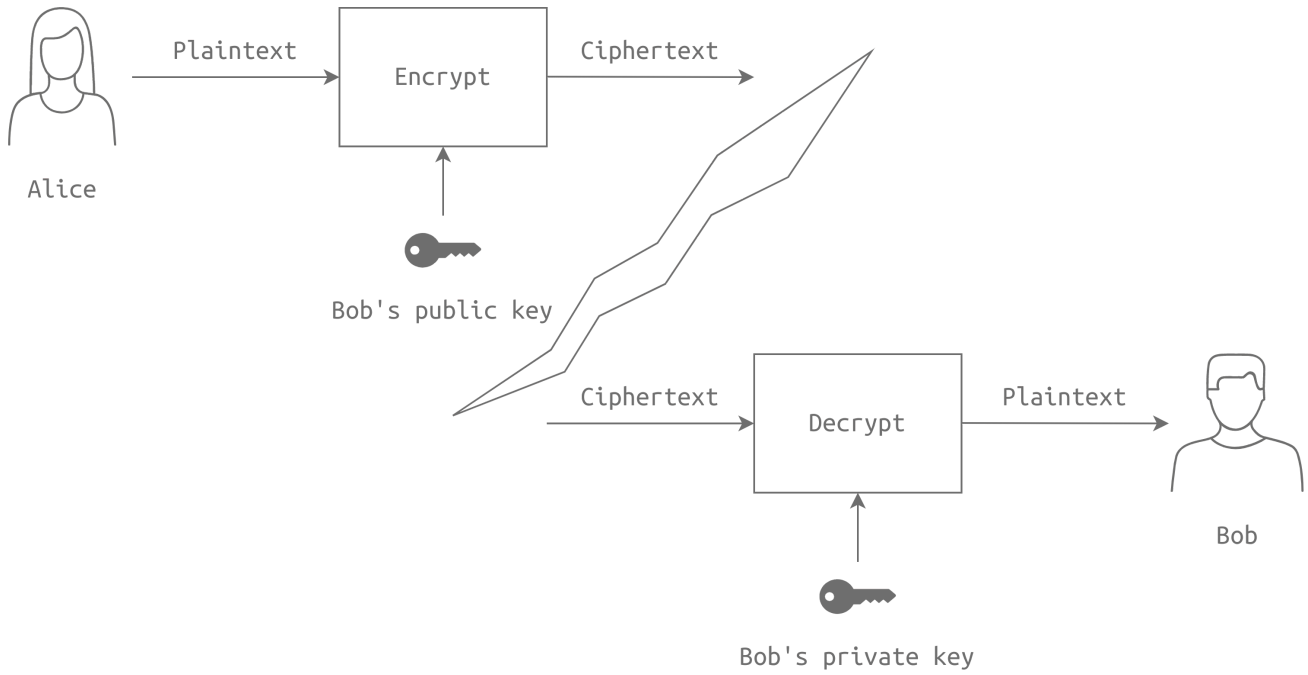
2. Kullanım Senaryoları: Hangi Anahtar Ne İçin?

Bu kısım çok karışır, o yüzden mantığını oturtalım:

A. Confidentiality (Gizlilik) - "Sadece O Okusun"

Amaç: Alice, Bob'a gizli bir mesaj göndermek istiyor.

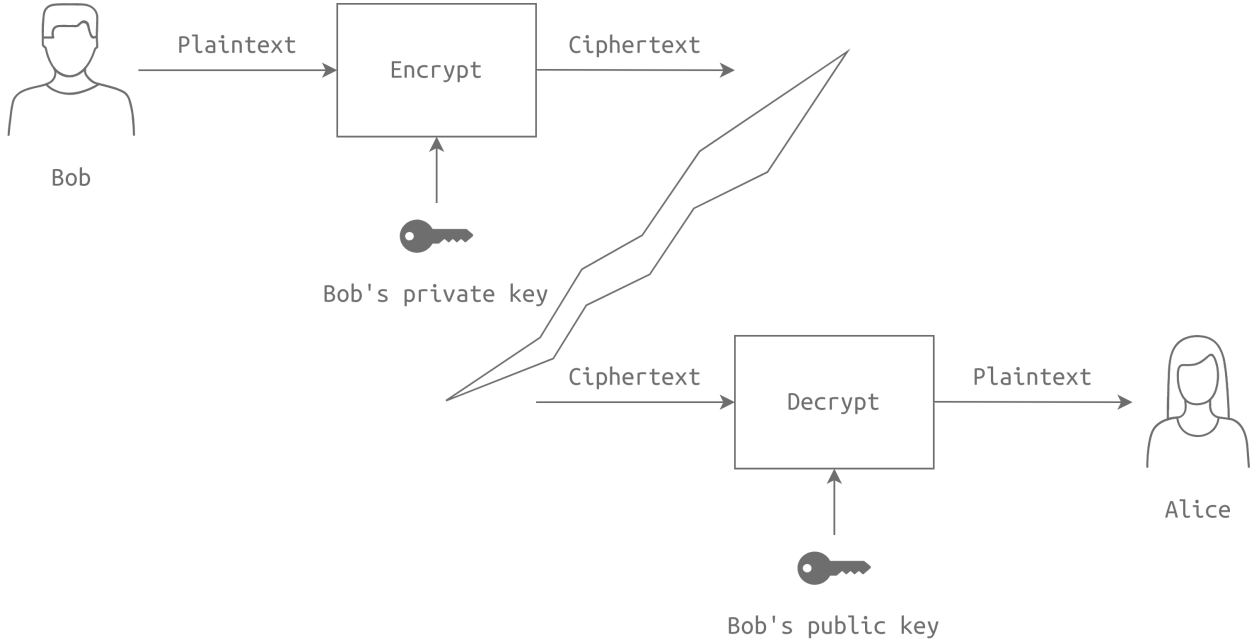
1. Alice, mesajı **Bob'un Public Key'i** ile şifreler.
2. Bu mesajı artık sadece **Bob'un Private Key'i** açabilir.
3. Sonuç: Mesaj yolda çalınsa bile kimse okuyamaz (Gizlilik sağlandı).



B. Integrity, Authenticity & Nonrepudiation (İmzalama) - "Benden Çıktığına Emin Olun"

Amaç: Bob bir duyuru yapıyor ve herkesin bu duyurunun gerçekten Bob'dan geldiğinden emin olmasını istiyor.

1. Bob, mesajı **kendi Private Key'i** ile şifreler.
2. Alıcılar, mesajı **Bob'un Public Key'i** ile çözmeyi dener.
3. Eğer mesaj başarıyla çözülüyorsa şu 3 şey kanıtlanmış olur:



- **Integrity (Bütünlük):** Mesaj yolda değiştirilmemiştir.
- **Authenticity (Kimlik Doğrulama):** Mesajı yazan kesinlikle Bob'dur (çünkü Private Key sadece onda).
- **Nonrepudiation (İnkâr Edilemezlik):** Bob daha sonra "Bunu ben yazmadım" diyemez.

Not: Asimetrik şifreleme matematiksel olarak çok işlem gücü gerektirir ve yavaştır. Gerçek hayatta (SSL/TLS gibi), asimetrik şifreleme sadece oturumun başında **simetrik anahtar güvenli paylaşmak** (Key Exchange) için kullanılır. Sonrasında hız için simetrik şifrelemeye geçilir (Hybrid Encryption).

3. RSA Algoritması (The Math Behind It)

RSA (Rivest, Shamir, Adleman), güvenliğini **Factorization (Çarpanlara Ayırma)** probleminin zorluğuna dayandırır. İki büyük asal sayıyı çarpmak kolaydır, ama çarpımı verip asalları bulmak (eğer sayılar yeterince büyükse) imkansıza yakındır.

Matematiksel Süreç

1. Anahtar Üretimi:

- İki rastgele büyük asal sayı seç: p ve q .
- Modül hesapla: $n = p \times q$
- n 'in totient fonksiyonunu hesapla: $\phi() = (p - 1)(q - 1)$
- Öyle bir e (public exponent) seç ki; $1 < e < \phi()$ olsun ve $\phi()$ ile aralarında asal olsun.
- Öyle bir d (private exponent) hesapla ki; $e \times d \equiv 1 \pmod{\phi()}$ olsun.

Sonuç:

- Public Key:** (n, e)
- Private Key:** (n, d)

2. Şifreleme (Encryption):

x mesajını şifrelemek için:

$$y = x^e \pmod{n}$$

3. Şifre Çözme (Decryption):

y şifreli metnini çözmek için:

$$x = y^d \pmod{n}$$

Güvenlik Notu: RSA'nın güvenli olması için p ve q sayılarının çok büyük (örn. 1024 bit, yani 300+ basamaklı) olması gerekir. Ayrıca rastgele sayı üretimi (Random Number Generation) güvenli olmalıdır.

4. Lab: OpenSSL ile RSA Pratiği

Şimdi bu teoriyi terminalde uygulayalım.

1. Private Key Üretimi

Bash


```
openssl genrsa -out private-key.pem 2048
```

- `genrsa` : RSA anahtarı üret.
- `2048` : Anahtar uzunluğu (bit cinsinden).
- Çıktı: `private-key.pem` dosyası.

2. Public Key Çıkarımı

Private key dosyasının içinden Public Key'i türetip ayıralım:

Bash

```
openssl rsa -in private-key.pem -pubout -out public-key.pem
```

- `-pubout` : Sadece Public Key kısmını dışarı ver.

3. Anahtar İçeriğini Okuma (Matematiksel Değerler)

Anahtarın içindeki p, q, e, d değerlerini görmek için:

Bash

```
openssl rsa -in private-key.pem -text -noout
```

Çıktı Analizi:

- `modulus` : sayısı.
- `publicExponent` : e sayısı (Genelde 65537 yani 0x10001 seçilir).
- `privateExponent` : d sayısı.
- `prime1` : p sayısı.
- `prime2` : q sayısı.

4. Şifreleme ve Şifre Çözme

Eğer elimizde **alıcının Public Key'i** (`public-key.pem`) varsa şifreleme yapabiliriz:

Şifreleme (Encrypt):

Bash

```
openssl pkeyutl -encrypt -in plaintext.txt -out ciphertext -inkey public-key.pem -pubin
```

- `pkeyutl` : Public Key Utilities aracı.
- `-encrypt` : Şifreleme modu.

- `-pubin` : Girdiğimiz anahtarın bir Public Key olduğunu belirtir.

Şifre Çözme (Decrypt):

Alıcı kendi **Private Key**'i ile şifreyi çözer:

Bash

```
openssl pkeyutl -decrypt -in ciphertext -inkey private-key.pem -out  
decrypted.txt
```

Diffie-Hellman Anahtar Değişimi

Asimetrik şifrelemenin ve modern güvenli iletişimin en kritik problemlerinden birini çözen, dahice bir algoritma: "**Güvensiz bir ortamda, herkesin bizi dinlediği bir odada, karşımdaki kişiyle kimsenin bilmediği ortak bir şifreyi nasıl belirlerim?**"

Bu algoritma **şifreleme (encryption)** yapmaz, **anahtar değişimi (key exchange)** yapar. Amaç, simetrik şifrelemede kullanacağımız o gizli anahtarı (Secret Key) oluşturmaktır.

1. Senaryo: Güvensiz Kanal (Insecure Channel)

- **Ortam:** Alice ve Bob konuşuyor.
- **Tehdit:** Hattı dinleyen bir saldırgan (Eve) var. Gönderilen her paketi okuyor.
- **Amaç:** Alice ve Bob öyle sayılar konuşmalı ki, konuşmanın sonunda ikisinin elinde **AYNI** sayı (Secret Key) oluşmalı ama hattı dinleyen Eve bu sayıyı hesaplayamamalı.

2. Nasıl Çalışır? (The Magic of Math)

Diffie-Hellman, modüler aritmetiğin (Modular Arithmetic) özelliklerini kullanır.

Kullanılan iki temel işlem:

1. **Üs Alma (Power):** x^p (x üzeri p).
2. **Mod Alma (Modulus):** $x \pmod{m}$ (x'in m'e bölümünden kalan).

Adım Adım Değişim Süreci

Alice ve Bob herkese açık iki sayı üzerinde anlaşarak başlar:

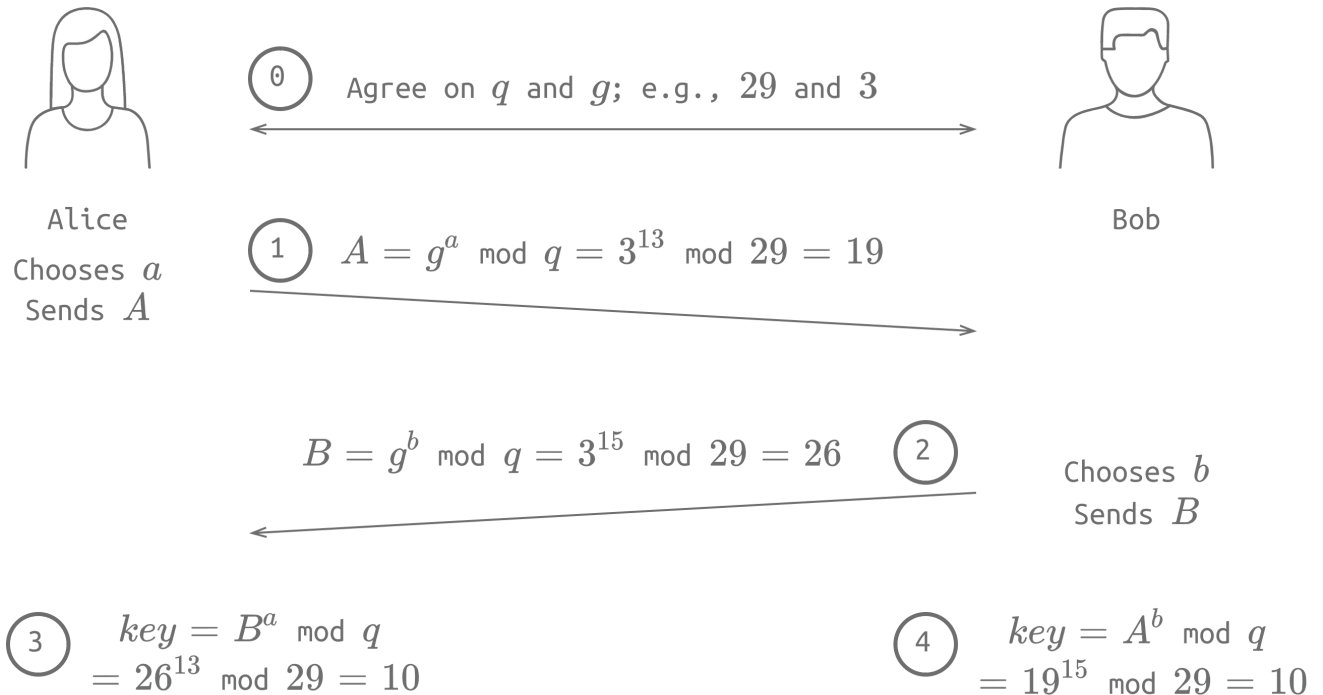
- **q (Prime Number):** Büyük bir asal sayı (Modül).
- **g (Generator):** q'dan küçük bir üreteç sayı.

Örnek Senaryo: $q = 29$, $g = 3$ olsun. (Saldırgan bunları biliyor).

Adım	Alice (A)	Bob (B)	Kanal (Herkes Görüyor)
1. Gizli Seçim	Rastgele bir gizli sayı seçer: $a = 13$	Rastgele bir gizli sayı seçer: $b = 15$	-
2. Public Hesap	$A = g^a \pmod{q}$ $A = 3^{13} \pmod{29} = 19$	$B = g^b \pmod{q}$ $B = 3^{15} \pmod{29} = 26$	-
3. Değişim	$A(19)$ sayısını Bob'a gönderir.	$B(26)$ sayısını Alice'e gönderir.	A=19, B=26 (Eve bunları gördü)
4. Ortak Sır	Gelen B ile kendi gizlisini kullanır: $Key = B^a \pmod{q}$ $26^{13} \pmod{29} = 10$	Gelen A ile kendi gizlisini kullanır: $Key = A^b \pmod{q}$ $19^{15} \pmod{29} = 10$	Eve hesaplayamıyor!

Sonuç:

- Alice: **10** buldu.
- Bob: **10** buldu.
- Saldırgan (Eve): Elinde $q = 29, g = 3, A = 19, B = 26$ var. Ancak a veya b gizli sayılarını bilmediği için **10** sonucuna ulaşamıyor.



3. Matematiksel Güvence (Neden Kırılmıyor?)

Bu sistemin güvenliği **Discrete Logarithm Problem** (Ayrık Logaritma Problemi) denilen bir zorluğa dayanır.

- **İşlem:** $3^{13} \pmod{29}$ işlemini yapmak bilgisayar için çok kolaydır (Easy).
- **Ters İşlem:** Sonucu (19) ve tabanı (3) bilip, "3 üzeri kaç mod 29'da 19 eder?" sorusunu ($\log_3 19 \pmod{29}$) çözmek, sayılar büyüdükçe imkansızlaşır (Hard/Infeasible).

Gerçek hayatta bu sayılar 13 veya 15 değil, **2048-bit** (yaklaşık 600 basamaklı) sayılardır.

- Örnekteki q sayısı 256-bit olsaydı bile, bu 10^{75} gibi devasa bir sayı olurdu.
- Bu büyüklükteki sayılarla gizli üsleri (a veya b) bulmak mevcut teknolojiyle binlerce yıl sürer.

4. Pratik Uygulama: OpenSSL ile Parametre Üretimi

Gerçek hayatta DH parametrelerini (P ve g) oluşturmak için `openssl` kullanırız.

Komut:

Bash

```
openssl dhparam -out dhparams.pem 2048
```

- `dhparam` : Diffie-Hellman parametresi üret.
- `2048` : Anahtar boyutu (Bit cinsinden). Ne kadar büyükse o kadar güvenli ama işlem o kadar yavaş.

İçeriği Okuma:

Üretilen dosyanın içindeki devasa Asal Sayıyı (P) ve Üretici (g) görmek için:

Bash

```
openssl dhparam -in dhparams.pem -text -noout
```

Çıktı Analizi:

Plaintext

```
DH Parameters: (2048 bit)
P:
    00:82:3b:9d... (Burada hex formatında yüzlerce satır sayı var)
G:    2 (0x2)
```

- **P (Prime):** Modüler aritmetikte kullanılan o devasa asal sayı (q).
- **G (Generator):** Üs alma işlemindeki taban sayı. Genelde 2 veya 5 gibi küçük bir sayıdır.

5. Kritik Zayıflık: Man-in-the-Middle (MitM)

Diffie-Hellman harika bir algoritmadır **ANCAK** tek başına kullanıldığında çok büyük bir açığı vardır: **Kimlik Doğrulama (Authentication) YOKTUR.**

Senaryo:

1. Alice "Merhaba ben Alice, parametrem A" der.
2. Saldırgan araya girer, Alice'e "Tamam ben Bob (yalan), parametrem X" der.
3. Saldırgan Bob'a döner, "Merhaba ben Alice (yalan), parametrem Y" der.
4. Alice, Saldırgan ile şifreli bir kanal kurar.
5. Bob, Saldırgan ile şifreli bir kanal kurar.

Sonuçta Saldırgan (MitM), Alice'ten gelen mesajı açar, okur, tekrar şifreleyip Bob'a gönderir. İkisi de güvenli iletişim kurduklarını sanarken aslında tüm trafiği saldırgan yönetmektedir.

Çözüm: Bu zayıflığı kapatmak için DH, **Dijital İmzalar (Digital Signatures)** veya **Sertifikalar** ile birlikte kullanılır (İlerideki görevlerde göreceğiz).

Hashing & HMAC

Bu notlar, bütünlüğün (integrity) nasıl sağlandığını, parolaların neden veritabanlarında düz metin olarak saklanmadığını ve HMAC yapısını anlamak için çıkarıldı. Kriptografinin en temel yapı taşlarından biridir.

1. Hashing Nedir? (The Fingerprint)

Cryptographic Hash Function (Özetleme Fonksiyonu): Boyutu ne olursa olsun (bir kelime veya 100 GB'lık bir dosya) herhangi bir veriyi alıp, çıktı olarak **sabit uzunlukta** (fixed size) bir değer döndüren algoritmadır. Bu çıktıya **Message Digest**, **Checksum** veya **Hash** denir.

Özellikler:

- **Tek Yönlüdür (One-way):** Hash değerinden orijinal veriye geri dönülemez. (Kıyma makinesinden çıkan etten ineği geri oluşturamazsın).
- **Sabit Uzunluk:** Girdi ne kadar büyük olursa olsun çıktı boyutu standarttır.
- **Deterministik:** Aynı girdi her zaman aynı çıktıyı verir.

Örnek: SHA256 (Secure Hash Algorithm 256)

- **Çıktı Boyutu:** 256 bit (32 Byte).
- **Gösterim:** Genelde Hexadecimal (16'lık taban) olarak yazılır.
 - 1 Hex karakter = 4 bit.

- 256 bit / 4 = **64 Hex karakter**.

Aşağıdaki terminal çıktısına dikkat et. 4 byte'lık dosya ile 5.2 GB'lık dosyanın hash uzunluğu **tamamen aynı**:

Bash

```
user@TryHackMe$ ls -lh
-rw-r--r-- 1 strategos strategos    4 abc.txt
-rw-r--r-- 1 strategos strategos 5.2G Win11_English_x64v1.iso

user@TryHackMe$ sha256sum *
c38bb113c89d8fec6475a9936411007c45563ecb7ce8acd5db7fb58c0872bda0  abc.txt
4bc6c7e7c61af4b5d1b086c5d279947357cff45c2f82021bb58628c2503eb64e  Win11_English_x64v1.iso
```

2. Neden Kullanıyoruz? (Use Cases)

A. Parola Saklama (Storing Passwords)

Veritabanında asla `password123` gibi düz metin (plaintext) saklanmaz. Bunun yerine hash'i saklanır.

- Veri sızıntısı olursa saldırgan sadece hash'leri görür (örn: `ef92b...`), gerçek parolaları göremez.
- (Not: *Pratikte "Salting" de eklenir, ileride göreceğiz*).

B. Değişiklik Tespiti (Integrity Check)

Hash fonksiyonlarının en kritik özelliği **Avalanche Effect (Çığ Etkisi)**'dir. Girdideki **tek bir bitlik** değişiklik, hash sonucunu tamamen değiştirir.

Örnek: `text1.txt` ("TryHackMe") ve `text2.txt` ("tryHackMe") dosyaları. Sadece "T" harfi "t" olmuş.

- ASCII'de 'T' = `0x54` (`01010100`)
- ASCII'de 't' = `0x74` (`01110100`)
- Fark sadece **1 bit**.

Bash

```
user@TryHackMe$ sha256sum text1.txt
f4616fd825a10ded9af58fbaee09f3e31751d15591f9323ea68b03a0e8ac3783  text1.txt

user@TryHackMe$ sha256sum text2.txt
9ffa3533ee33998aeb1df76026f8031c8af6ccabd8393eca002d5b7471a0b536  text2.txt
```

Sonuç: Hash'ler tamamen alakasız. Bu sayede dosya transferinde bir bit bile bozulsa veya biri dosyayı değiştirse (tampering) anında anlarız.

3. Algoritmaların Durumu (Secure vs Broken)

Hangi algoritmayı kullanmalıyız?

- **Güvenli (Secure):** SHA256, SHA384, SHA512, SHA224, RIPEMD160.
- **Kırılmış (Broken):** MD5, SHA-1.

"Kırılmış" (Broken) Ne Demek?

Bir hash algoritması için en büyük kâbus **Collision (Çakışma)** durumudur.

- Eğer iki *farklı* dosya **aynı hash değerini** üretiyorsa, o algoritma kırılmıştır.
- MD5 ve SHA-1'de saldırganlar, orijinal dosyayla aynı hash'e sahip zararlı bir dosya üretebiliyorlar. Bu yüzden güvenlik gerektiren yerlerde asla kullanılmazlar.

4. HMAC (Hash-based Message Authentication Code)

Standart hash (SHA256 vb.) sadece dosyanın değişmediğini (integrity) kanıtlar ama **kimden geldiğini** kanıtlamaz. Herkes `sha256sum` çalıştırabilir.

Hem **bütünlük (integrity)** hem de **kimlik doğrulama (authentication)** istiyorsak **HMAC** kullanırız.

- **Mantık:** Hash Fonksiyonu + Gizli Anahtar (Secret Key).

Nasıl Çalışır? (RFC2104)

HMAC, veriyi doğrudan hash'lemek yerine, bir gizli anahtar ve özel dolgu (padding) değerleri ile karıştırarak hash'ler.

Bileşenler:

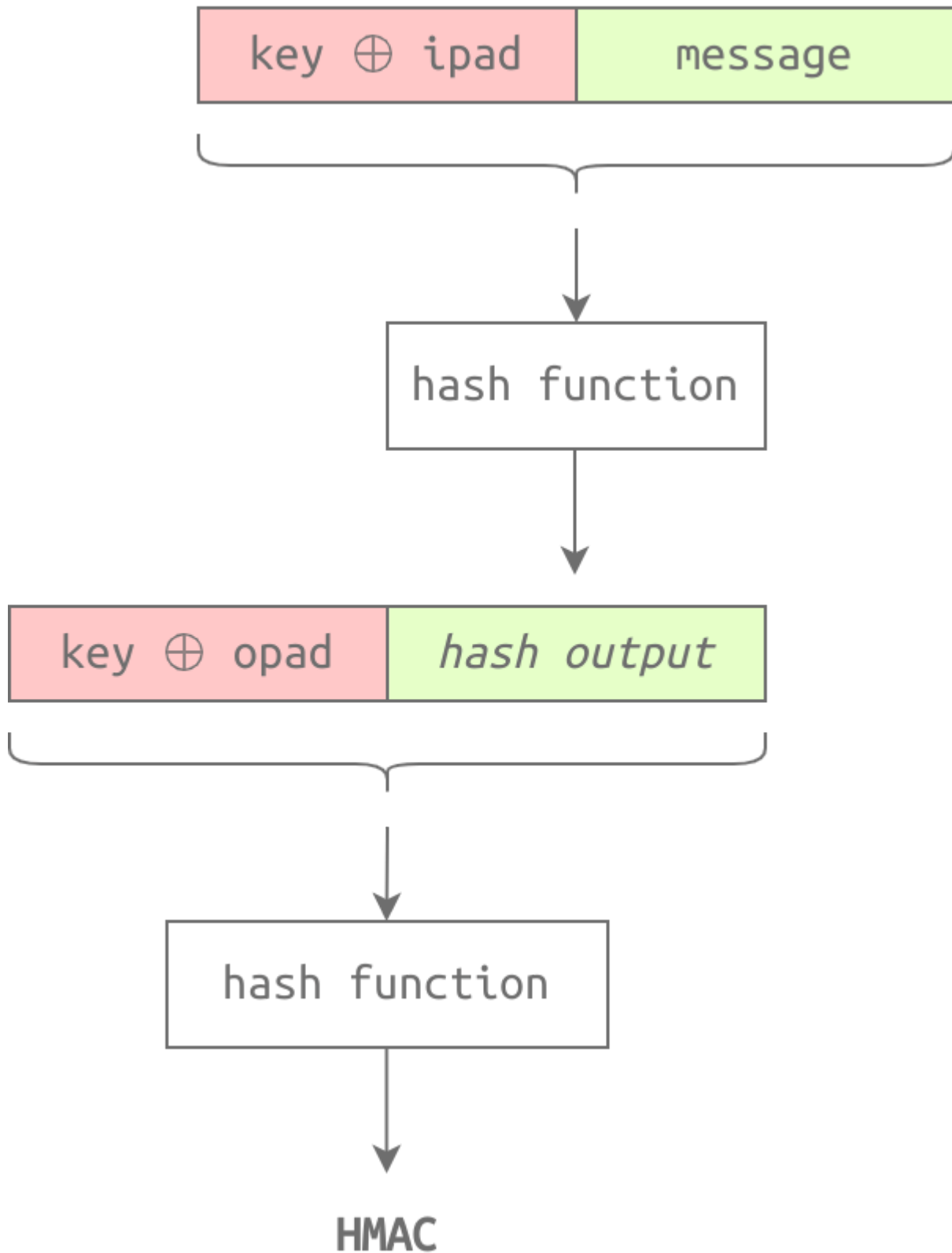
1. **Secret Key (K):** Sadece gönderici ve alıcı bilir.
2. **ipad (Inner Pad):** Sabit byte (`0x36`).
3. **opad (Outer Pad):** Sabit byte (`0x5C`).

Matematiksel Formül:

$$HMAC(K, text) = H(K \oplus opad, H(K \oplus ipad, text))$$

Süreç (Adım Adım):

1. **Anahtar Hazırlığı:** Anahtarın boyutu ayarlanır, gerekirse sıfır eklenir.
2. **İç Döngü:** Anahtar XOR `ipad` yapılır, mesaja eklenir ve Hash alınır.
3. **Dış Döngü:** Anahtar XOR `opad` yapılır, önceki hash sonucuna eklenir ve tekrar Hash alınır.



Linux'ta HMAC Kullanımı

Aynı mesaj (`message.txt`), farklı anahtarlarla (`s!Kr37` ve `1234`) HMAC işlemine sokulduğunda sonuç tamamen farklı çıkar. Bu, mesajın o anahtara sahip kişiden geldiğini kanıtlar.

Bash


```
# Anahtar: s!Kr37
user@TryHackMe$ hmac256 s!Kr37 message.txt
3ec65b7e80c5bf2e623e52e0528f1c6a74f605b10616621ba1c22a89fb244e65
message.txt

# Anahtar: 1234
user@TryHackMe$ hmac256 1234 message.txt
4b6a2783631180fca6128592e3d17fb5bff6b0e563ad8f1c6afc1050869e440f
message.txt

# Alternatif Komut (sha256hmac)
user@TryHackMe$ sha256hmac message.txt --key s!Kr37
3ec65b7e80c5bf2e623e52e0528f1c6a74f605b10616621ba1c22a89fb244e65
message.txt
```

Özet:

- **Hash:** "Dosya değişmedi." (Integrity)
- **HMAC:** "Dosya değişmedi VE anahtarı bilen doğru kişiden geldi." (Integrity + Authentication)

PKI (Public Key Infrastructure) ve SSL/TLS

Önceki notlarda (Diffie-Hellman) güvenli olmayan bir hat üzerinden nasıl anahtar paylaşacağımızı gördük. Ancak orada devasa bir açık bıraktık: **Kimlik Doğrulama (Authentication)**.

Karşımdakinin gerçekten Alice mi yoksa araya giren Mallory mi olduğunu bilmiyorsam, şifreli konuşmamın bir anlamı kalmaz. İşte PKI (Açık Anahtar Altyapısı) tam olarak bu sorunu çözer.

1. Sorun: Man-in-the-Middle (MitM) Saldırısı

Diffie-Hellman tek başına MitM saldırılarına karşı savunmasızdır. Saldırgan (Mallory), Alice ve Bob arasına girer.

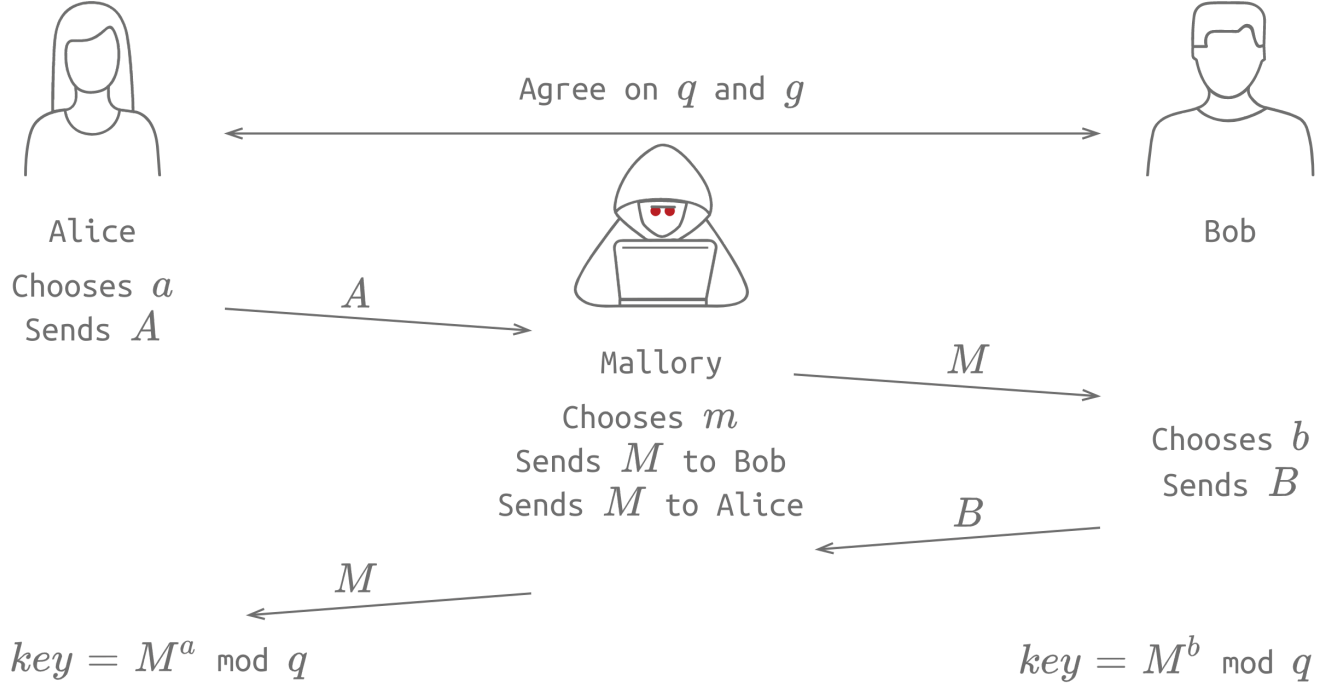
Saldırı Senaryosu (Adım Adım)

Alice ve Bob, q ve g değerlerinde anlaştı (bunlar açık).

1. **Alice:** Gizli a 'sını seçer, A 'yı hesaplar ve Bob'a gönderir.
2. **Mallory (Saldırgan):** Hattı dinler, A 'yı yakalar. Kendi gizli m 'sini seçer, M 'yi hesaplar ve Bob'a "**Ben Alice**" diyerek M 'yi gönderir.
3. **Bob:** Gelen M 'yi Alice'ten geldi sanır. Kendi b 'sini seçer, B 'yi hesaplar ve Alice'e gönderir.
4. **Mallory:** Hattı dinler, B 'yi yakalar. Alice'e "**Ben Bob**" diyerek yine kendi M 'sini gönderir.

Sonuç:

- **Alice:** M ve a ile bir anahtar hesaplar (Mallory ile şifreli hat kurar).
- **Bob:** M ve b ile bir anahtar hesaplar (Mallory ile şifreli hat kurar).
- Alice ve Bob güvenli iletişim kurduklarını sanırlar ama aslında tüm trafiği Mallory okur (decrypt eder), isterse değiştirir ve karşı tarafa kendi anahtarıyla şifreleyip (re-encrypt) iletir.



2. Çözüm: PKI ve Sertifikalar

Bu sorunu çözmek için **Güvenilir Üçüncü Taraf (Trusted Third Party)** mekanizmasına ihtiyacımız var. İnternette buna **Certificate Authority (CA)** denir.

HTTPS ve Kilit İkonu

Bir web sitesine (örneğin `example.org`) girdiğinde adres çubuğundaki kilit ikonu şunu söyler:

1. Bağlantı şifrelidir (SSL/TLS).
2. Bu site **gerçekten** `example.org` 'dur (Kimlik Doğrulama).

Bunu sağlayan şey **Dijital Sertifikadır**. Tarayıcın, `example.org` 'un sertifikasının güvenilir bir CA (örneğin DigiCert Inc.) tarafından imzalandığını görür. Tarayıcının içinde DigiCert'in Public Key'i gömülü olduğu için imzayı doğrulayabilir.

3. Sertifika İmzalama Süreci (CSR ve CA)

Gerçek dünyada bir web sunucusu kurarken SSL sertifikası almak için şu adımları izleriz:

1. **CSR (Certificate Signing Request) Oluřturma:** Kendi sunucumuzda bir Private Key ve Public Key üretiriz. Public Key'i ve kimlik bilgilerimizi (Domain, Őehir, Ülke vb.) bir dosyada toplarız. Buna CSR denir.
2. **CA'ya Gönderme:** CSR dosyasını CA'ya (DigiCert, Let's Encrypt vb.) göndeririz.
3. **İmzalama:** CA, bizi doğrular ve sertifikamızı kendi Private Key'i ile imzalar.
4. **Güven:** Tarayıcılar CA'ya güvendiğı için, CA'nın imzaladığı sertifikaya da güvenir.

OpenSSL ile CSR Oluřturma

Bu komut CTF'lerde ve sunucu yönetiminde çok çıkar.

Bash

```
openssl req -new -nodes -newkey rsa:4096 -keyout key.pem -out cert.csr
```

Parametrelerin Anlamı (Ezberle):

- `req -new` : Yeni bir sertifika imzalama isteğı (CSR) oluřtur.
- `-nodes` : "No DES". Private Key'i řifrelemeden (passphrase olmadan) kaydet. (Sunucu her restart olduğunda řifre girmemek için).
- `-newkey` : Yeni bir Private Key de üret (önceden yoksa).
- `rsa:4096` : RSA algoritması kullan ve anahtar boyutu 4096 bit olsun.
- `-keyout key.pem` : Oluřan Private Key'i bu dosyaya kaydet. (**ÇOK GİZLİ TUTULMALI**)
- `-out cert.csr` : Oluřan CSR'ı bu dosyaya kaydet. (Bu dosya CA'ya gönderilecek).

Komutu çalıştırdıktan sonra terminal sana Ülke (UK), Őehir (London), Organizasyon adı gibi sorular soracaktır. Bu bilgiler sertifikanın içine gömülür.

4. Self-Signed Certificate (Kendinden İmzalı Sertifika)

Test ortamlarında veya internal (řirket içi) ağlarda para verip CA'dan imza almayız. Sertifikayı **kendi kendimize** imzalarız.

Tarayıcılar buna güvenmez ve "Güvenli Değil" uyarısı verir ama řifreleme yine de çalışır.

Self-Signed Sertifika Komutu:

Bash

```
openssl req -x509 -newkey -nodes rsa:4096 -keyout key.pem -out cert.pem -sha256 -days 365
```

Farklı Parametreler:

- `-x509` : Bir CSR değıl, doğrudan X.509 formatında bir **Sertifika** üretmek istediğimizi belirtir.

- `-days 365` : Sertifika 1 yıl (365 gün) geçerli olsun.
- `-sha256` : İmza algoritması olarak SHA-256 kullan (güvenlik için).
- `Çıktı:` `cert.pem` (Kullanıma hazır sertifika).

5. Sertifikayı İnceleme (Inspection)

Elimizdeki bir sertifika dosyasının (`cert.pem`) kime ait olduğunu, kimin imzaladığını, geçerlilik süresini görmek için:

Bash

```
openssl x509 -in cert.pem -text
```

- `-in` : Girdi dosyası.
- `-text` : Sertifikayı okunabilir metin formatında dök.

Bu komutun çıktısında şunlara dikkat et (CTF soruları buradan gelir):

- **Issuer:** Sertifikayı kim imzalamış? (Self-signed ise Subject ile aynıdır).
- **Subject:** Sertifika kime verilmiş? (CN=Common Name, domain adı).
- **Validity:** Not Before ve Not After tarihleri.
-

Password Authentication & Storage (Parola Doğrulama ve Saklama)

Bu bölümde, parolaların veritabanında **nasıl saklanması gerektiğini** (Data at Rest) ve saldırganların veritabanını ele geçirse bile parolaları okumasını nasıl engelleyeceğimizi inceliyoruz.

Önceki konuda (SSL/TLS) veriyi ağ üzerinde giderken (**Data in Transit**) korumuştuk. Şimdi sıra veritabanında dururken korumakta.

1. Parola Saklama Yöntemlerinin Evrimi

Bir web uygulamasında parolaları saklamanın "En Kötü"den "En İyi"ye doğru sıralaması şöyledir:

Seviye 0: Plaintext (Düz Metin) - ❌ ASLA YAPMA

Kullanıcı adı ve parolayı veritabanına olduğu gibi yazmak.

- **Senaryo:** Alice'in parolası "qwerty". Veritabanına `alice | qwerty` olarak kaydedilir.
- **Risk:** Veritabanı sızarsa (Data Breach), saldırgan **hiçbir efor sarf etmeden** herkesin parolasını görür. Bu bir felakettir.

Seviye 1: Hashing (Özetleme) - ⚠️ YETERSİZ

Parolayı hash fonksiyonundan (MD5, SHA256 vb.) geçirip saklamak.

- **Senaryo:** `alice | MD5("qwerty") → alice | d8578edf8458...`
- **Mantık:** Hash tek yönlüdür, geri döndürülemez. Saldırgan hash'i görse de parolayı bilemez... SANIYORDUK.
- **Zayıflık (Rainbow Tables):** Saldırganlar yaygın parolaların hash'lerini önceden hesaplayıp devasa tablolarda (Rainbow Table) saklarlar.
 - Saldırgan veritabanında `d857...` hash'ini görür.
 - Kendi tablosunda bu hash'i aratır.
 - Tablo ona karşılığının "qwerty" olduğunu söyler. **Hash kırıldı.**

Seviye 2: Salting (Tuzlama) - ✅ GÜVENLİ

Rainbow Table saldırısını bitiren yöntemdir. Her kullanıcı için rastgele üretilen bir **Salt** (Tuz) değeri eklenir.

- **Formül:** `Hash(Password + Salt)`
- **İşleyiş:**
 1. Alice "qwerty" parolasını girer.
 2. Sistem rastgele bir salt üretir: `12742`.
 3. Veritabanına şu kaydedilir: `Hash("qwerty" + "12742")` ve Salt değeri `12742`.
- **Neden Güvenli?**
 - Alice ve Bob'un parolası aynı ("qwerty") olsa bile, Salt değerleri farklı olduğu için (Alice: `12742`, Bob: `8861`) ortaya çıkan Hash **tamamen farklı** olur.
 - Saldırganın Rainbow Table'ı işe yaramaz çünkü tablosunda "qwerty"nin hash'i vardır ama "qwerty12742"nin hash'i yoktur. Saldırgan her kullanıcı için sıfırdan hesaplama yapmak zorunda kalır.

Seviye 3: Key Derivation (PBKDF2) - 🏆 ALTIN STANDART

Salting harika olsa da, modern GPU'lar saniyede milyarlarca hash deneyebilir (Brute-Force). Bunu yavaşlatmak için **PBKDF2** (Password-Based Key Derivation Function 2) gibi fonksiyonlar kullanılır.

- **Mantık (Iterations):** Hash işlemini sadece bir kez değil, binlerce kez (örneğin 100.000 iterasyon) yapar.
- **Amaç:** Yasal kullanıcı giriş yaparken 0.1 saniye bekler (fark etmez), ama saldırgan 1 milyar deneme yapmak isterse her denemede 0.1 saniye beklemek zorunda kalır. Bu da saldırıyı pratik olarak imkansız kılar (Yıllar sürer).

Özet:

- **Plaintext:** Kapıyı açık bırakmak.

- **Hashing:** Kapıyı kilitlemek ama anahtarı paspasın altına koymak (Rainbow Table bulur).
- **Salting:** Her kilide farklı anahtar uydurmak (Rainbow Table çalışmaz).
- **PBKDF2:** Anahtarı çevirmeyi 100.000 kat zorlaştırmak (Brute-Force çok zaman alır).

HTTPS & Login Flow: Putting It All Together (Örnek Senaryo)

Bu son bölümde, öğrendiğimiz tüm kavramları (PKI, Digital Signature, Hashing, Symmetric/Asymmetric Encryption, Salting) gerçek bir HTTPS oturum açma senaryosunda birleştiriyoruz.

Bir web sitesine girip kullanıcı adı/parola yazdığımızda arka planda saniyeler içinde şu 4 kritik aşama gerçekleşir:

Adım 1: Sertifika Doğrulama (The ID Check)

Kullanılan Teknoloji: PKI, Digital Signatures, Hashing, Asymmetric Encryption.

1. **İstek:** Client (Tarayıcı), sunucudan SSL/TLS Sertifikasını ister.
2. **Gönderim:** Sunucu sertifikasını gönderir.
3. **Doğrulama (Validation):** Tarayıcı bu sertifikanın sahte olup olmadığını kontrol eder.
 - **İmza Mantiğı:** Sertifikayı imzalayan CA (Certificate Authority), sertifikanın Hash'ini alır ve kendi **Private Key**'i ile şifreler. Bu şifreli hash, sertifikaya eklenir (İmza).
 - **Kontrol:** Tarayıcıda CA'nın **Public Key**'i zaten yüklüdür.
 1. Tarayıcı, imzayı CA'nın Public Key'i ile çözer (Decrypt) → Orijinal Hash .
 2. Tarayıcı, sertifikanın kendisi hash'ler → Hesaplanan Hash .
 3. Eğer Orijinal Hash == Hesaplanan Hash ise, sertifika geçerlidir ve değiştirilmemiştir.
 - **Sonuç:** Sunucunun kimliği doğrulanır (Authentication).

Adım 2: Handshake (El Sıkışma & Anahtar Değişimi)

Kullanılan Teknoloji: Asymmetric Encryption (Key Exchange), Symmetric Encryption.

1. Sertifika doğrulandıktan sonra, Client ve Server güvenli bir şekilde konuşmak için anlaşmalıdır.
2. **Amaç:** İletişimi şifrelemek için ortak bir **Secret Key** (Simetrik Anahtar) üzerinde anlaşmak.
3. **Yöntem:** Asimetrik şifreleme (RSA veya Diffie-Hellman) kullanılarak bu ortak anahtar güvenli bir şekilde oluşturulur.
4. **Neden?** Çünkü asimetrik şifreleme çok yavaştır. Veri transferi için hızlı olan simetrik şifrelemeye geçilmelidir.

Adım 3: Güvenli Veri İletişimi (Data in Transit)

Kullanılan Teknoloji: Symmetric Encryption (AES vb.).

- Artık hem Client hem Server aynı **Secret Key**'e sahiptir.
- Kullanıcı adı ve parola tarayıcıdan çıktığı anda bu anahtarla şifrelenir (Encrypted Tunnel).
- Ağ üzerindeki hiç kimse (ISP, Hacker, Eve) bu veriyi okuyamaz (Confidentiality).

Adım 4: Kimlik Doğrulama (Data at Rest)

Kullanılan Teknoloji: Hashing, Salting.

- Sunucu, şifreli tünelden gelen kullanıcı adı ve parolasını (Plaintext olarak) alır.
- Veritabanı Kontrolü:** Sunucu, gelen parolayı veritabanındakiyle nasıl kıyaslar?
 - Veritabanında parola **ASLA** açık durmaz. Salt + Hashed Password durur.
- İşlem:**
 - Sunucu, o kullanıcıya ait **Salt** değerini veritabanından çeker.
 - Gelen parolaya bu tuzu ekler: Gelen Parola + Salt .
 - Bunun Hash'ini alır (SHA256/PBKDF2).
 - Çıkan sonucu veritabanındaki kayıtlı Hash ile karşılaştırır.
- Eşleşirse giriş başarılıdır.

Özet Tablo: Kim Ne Yapıyor?

Aşama	Teknoloji	Amaç
Bağlantı Kurma	PKI & Digital Signature	Sunucunun "Ben example.com'um" ispatı. (Authentication)
Anahtar Değişimi	Asymmetric Encryption	Ortak şifreleme anahtarını güvenli bir şekilde belirleme.
Veri Transferi	Symmetric Encryption	Veriyi hızlı ve gizli taşıma. (Confidentiality)
Giriş (Login)	Hashing & Salting	Parolayı güvenli saklama ve doğrulama.

Sonuç ve Kapanış

Bu oda (room) ile kriptografinin sadece teorik matematik değil, internetin güvenliğini sağlayan pratik bir zincir olduğunu gördük. Bir zincir en zayıf halkası kadar güçlüdür; bu yüzden modern sistemlerde **PKI + Simetrik Şifreleme + Hashing + Salting** hepsi bir arada kullanılır.